CENTRO REGIONAL UNIVERSITARIO CÓRDOBA - IUA

Trabajo Final

“Mini-Compilador de C ”

Desarrollo de Herramientas de Software – Ingeniería en Informática

Profesor

Eschoyez, Maximiliano A.

Alumno

Contreras, Joaquin

jcontreras755@alumnos.iua.edu.ar

22 de Febrero de 2026

Índice

Índice 1
Introducción y Problemática Abordada 3
Características Principales 3
Herramientas Utilizadas: 3
Desarrollo de la Solución Propuesta 4
Arquitectura del Compilador 4
Diagrama de módulos 4
Patrones de diseño utilizados 4
Flujo de ejecución 5
Análisis Léxico 5
Categorías de Tokens 5
Análisis Sintáctico 6
Tabla de Símbolos 6
Gestión de pila de contextos 6
SymbolTableManager (Facade) 7
Funciones y modelo de pila de parámetros 7
Análisis Semántico 7
Validaciones implementadas 8
Gestión de Errores y Advertencias 8
Categorías 8
Flujo de errores 8
Generacion de Codigo Intermedio (TAC) 9
Formato de instrucciones TAC 9
Generación para expresiones 9
Generación para while 10
Generación para for 10
Generación para if-else 10
Generación para funciones 11
Optimización de código TAC 11
Plegado de constantes 12
Propagación de Copias 12
Eliminación de Código muerto 13
Simplificación Algebraica 13
Iteración hasta punto fijo 13
Reporte y comparación 14
Resumen de Archivos del Proyecto 14
Conclusión 14
Referencias 15

Introducción y Problemática Abordada

El objetivo de este Trabajo Final para la materia "Desarrollo de Herramientas de Software" consiste en la implementación de un compilador para un subconjunto del lenguaje C. La problemática principal está en transformar un código fuente de alto nivel, legible por humanos, en una representación intermedia que permita verificar la corrección gramatical y lógica del programa, preparándolo para fases posteriores de compilación.

Características Principales

• Análisis léxico y sintáctico generado por *ANTLR4*

• Tabla de símbolos con soporte de *scopes* anidados (pila de contextos)

• Validación semántica: tipos, inicialización, declaración, doble declaración

• Generación de *TAC* para todas las estructuras del lenguaje

• Parámetros de funciones usando modelo de pila (*push/pop*)

• Optimización de *TAC* con 4 pasadas: *constant folding, copy propagation, dead code elimination, algebraic simplification*

• Bloqueo de generación de *TAC* cuando existen errores (advertencias no bloquean)

• Comparación antes/después de la optimización con métricas de reducción

Herramientas Utilizadas:

- ***Python***: Lenguaje anfitrión para la lógica del compilador.
- ***ANTLR4 (Another Tool for Language Recognition):*** Herramienta utilizada para generar el analizador léxico (*Lexer*) y sintáctico (*Parser*) a partir del archivo de *gramática .g4* .
- ***Git***: Para el control de versiones y gestión del proyecto.

Desarrollo de la Solución Propuesta

Arquitectura del Compilador

Para abordar la solución, el compilador sigue una arquitectura modular con una separación clara de responsabilidades. Cada fase del análisis está implementada en un módulo independiente que se comunica a través de interfaces bien definidas.

Diagrama de módulos

```
App.py  (orquestador)
  |-- compiladorLexer.py   (ANTLR4 - análisis léxico)
  |-- compiladorParser.py  (ANTLR4 - análisis sintáctico)
  |-- Escucha.py           (Listener - análisis semántico)
  |     |-- ErrorManager.py         (singleton - gestión de errores)
  |     |-- SemanticValidator.py    (validación de tipos)
  |     |-- SymbolTableManager.py   (operaciones sobre TS)
  |     |-- StatsCollector.py       (estadísticas)
  |     '-- tablaDeSimbolos.py      (TS, Contexto, Variable, Funcion)
  |-- Caminante.py         (Visitor - recorrido del árbol)
  |-- IntermediateCodeGenerator.py  (Visitor - generación TAC)
  '-- TACOptimizer.py      (optimización de TAC)
```

Patrones de diseño utilizados

- ***Singleton*:** *ErrorManager* y Tabla de símbolos usan instancia única global.
- ***Listener*:** *Escucha.py* implementa *compiladorListener* para recibir eventos enter/exit de cada nodo del AST durante el recorrido.
- ***Visitor*:** *IntermediateCodeGenerator.py* y *Caminante.py* implementan *compiladorVisitor* para recorrer el árbol de sintaxis abstracta (AST) de forma controlada, retornando valores.
- ***Facade*:** *SymbolTableManager* abstrae las operaciones sobre la tabla de símbolos.

Flujo de ejecución

```
1. ErrorManager.reset()           — Reiniciar estado
2. Lexer → tokens                 — Análisis léxico
3. Parser → AST                   — Análisis sintáctico
4. Walker + Escucha → semántico   — Recorrido listener
5. if tiene_errores(): STOP       — Bloqueo en errores
6. Caminante.visit(AST)           — Recorrido visitor
7. ICG.visit(AST) → TAC original  — Generación de TAC
8. TACOptimizer → TAC optimizado  — Optimización
9. Reporte de comparación         — Métricas
```

Análisis Léxico

El análisis léxico se define en la gramática compilador.g4 y es generado automáticamente por ANTLR4. En esta etapa, el lexer se encarga de establecer los tokens (identificadores, palabras reservadas, operadores) y las reglas de producción que definen la estructura jerárquica del lenguaje (programas, bloques, instrucciones, expresiones).

Categorías de Tokens

Categoría	Tokens	Ejemplo
Agrupadores	PA PC LLA LLC CA CC PYC COMA	( ) { } [ ] ; ,
Aritméticos	SUMA RESTA MULT DIV MOD	+ - * / %
Asignación compuesta	MASIG RESIG MULASIG DIVASIG MODASIG	+= -= *= /= %=
Comparación	MENOR MAYOR MENOREQ MAYOREQ EQUAL NEQ	< > <= >= == !=
Incremento/Decremento	INCREMENTO DECREMENTO	++ --
Literales	NUMERO	42 0 123
Tipos	INT DOUBLE	int double
Control	IF ELSE FOR WHILE RETURN	if else for while return
Identificadores	ID	x foo miVar_1

Análisis Sintáctico

La gramática (*compilador.g4*) define un lenguaje imperativo con tipos estáticos basados en *C*. Mediante *ANTLR4*, se generó un analizador sintáctico (*Parser LL(*)*) que procesa los tokens y construye un árbol de sintaxis concreta (*Parse Tree*).

En esta etapa, el *Parser* se encarga de verificar la correcta estructura del código fuente, validando específicamente:

- El balanceo de llaves {}, corchetes [] y paréntesis ().
- La correcta terminación de instrucciones con punto y coma ';'.
- La estructura, jerarquía y precedencia válida de las operaciones aritméticas y lógicas.
- La correcta formación gramatical de las estructuras de control de flujo (*if, while, for*), así como la declaración y el llamado a funciones de usuario.

Adicionalmente, para cumplir con los requisitos de verificación, se implementó un manejador personalizado (*ErrorManager).* Este módulo intercepta los errores sintácticos detectados por *ANTLR4* y los reporta de forma clara, deteniendo las siguientes fases de la compilación si el código fuente resulta ser invalido.

Tabla de Símbolos

La tabla de símbolos (*TS*) es una estructura fundamental del compilador que almacena información sobre todos los identificadores declarados, osea, variables y funciones.

Gestión de pila de contextos

Los contextos (o *scopes*) se implementan como una pila (lista) de objetos Contexto. Al entrar a un bloque (*función, while, for, if*), se hace push de un nuevo contexto. Al salir, se hace pop. La búsqueda de símbolos recorre la pila desde el tope hacia la base, implementando el *shadowing* de variables *(*cuando una variable declarada en un contexto interno tiene el mismo nombre que una variable declarada en un contexto externo, lo que significa que, dentro de ese contexto, se utilizará la variable interna y no la externa).

SymbolTableManager (Facade)

SymbolTableManager proporciona una capa de abstracción sobre la *TS* con métodos de alto nivel:

- agregar_variable(nombre, tipo, línea, inicializado)
- agregar_funcion(nombre, tipo_retorno, parámetros, línea) — marca la función como inicializada
- buscar_simbolo(nombre) — busca en todos los scopes
- buscar_en_contexto_actual(nombre) — solo scope actual
- marcar_como_usado(nombre)

- marcar_como_inicializado(nombre, línea)

Funciones y modelo de pila de parámetros

Cuando se declara una función, el compilador sigue este proceso:

1. Registra la función en el *scope* PADRE (permite recursión y visibilidad externa).
2. Abre un *scope* HIJO (*push* de contexto) para el cuerpo de la función.
3. Agrega los parámetros al *scope* HIJO como variables inicializadas (*push* a la pila).
4. Al salir de la función (*exitFuncion*), cierra el *scope* HIJO (*pop),* eliminando parámetros y variables locales.

Análisis Semántico

El análisis semántico se realiza en *Escucha.py (Listener)* con apoyo de *SemanticValidator.py.* Se ejecuta durante el recorrido del AST por el *ParseTreeWalker*.

Validaciones implementadas

Validación	Método	Tipo
Variable no declarada	validar_variable_declarada	Error
Variable no inicializada	validar_variable_inicializada	Error
Doble declaración	validar_doble_declaracion	Error
Tipos incompatibles	validar_tipo_compatible	Error
Variable no usada	validar_variables_no_usadas	Advertencia
Función no declarada	enterFactor (con PA)	Error
Módulo con no-enteros	_validar_operacion_binaria	Error

Las variables declaradas pero nunca usadas se reportan como advertencias, no como errores. Esto permite generar código de Tres Direcciones (*TAC)* para programas que tienen variables sin usar – un caso común durante el desarrollo.

Gestión de Errores y Advertencias

ErrorManager es un *singleton* que centraliza la recolección y reporte de todos los errores y advertencias del compilador.

Categorías

Categoría	Método	Bloquea TAC?
Error sintáctico	reportar_error_sintactico(línea, msg)	Sí
Error semántico	reportar_error_semantico(línea, msg)	Sí
Advertencia	reportar_advertencia(línea, msg)	No

Flujo de errores

- *ErrorManager.reset()* se llama al inicio de App.py, ANTES del *parsing*.
- Los Errores Sintacticos se capturan con *CustomErrorListener* durante el parse.
- *EnterPrograma* NO llama reiniciar() – así se preservan los errores sintácticos
- Los Errores Semánticos se agregan durante el *walk* del *listener*.
- *Tiene_errores()* verifica si hay errores (no cuenta las advertencias).
- Si hay errores → se bloquea la generación de *TAC* con un mensaje informativo.

Generacion de Codigo Intermedio (*TAC)*

El código de tres direcciones *(TAC)* es una representación intermedia donde cada instrucción tiene como máximo un operador y tres operandos. Se genera mediante *IntermediateCodeGenerator.py,* que implementa el patrón Visitor.

Formato de instrucciones *TAC*

Tipo	Formato	Ejemplo
Asignación simple	var = valor	x = 5
Operación binaria	t = a op b	t0 = x + 3
Declaración	DECLARE var	DECLARE y
Salto condicional	if not cond goto L	if not t0 goto L1
Salto incondicional	goto L	goto L0
Etiqueta	L:	L0:
Parámetro	param valor	param x
Llamada	t = call func	t0 = call suma
Retorno	return valor	return t0
Función inicio	FUNC nombre:	FUNC factorial:
Función fin	END FUNC nombre	END FUNC factorial
Pop parámetro	pop nombre	pop n

Generación para expresiones

Las expresiones se evalúan *bottom-up*. Cada subexpresión genera un temporal (t0, t1, ...) que se usa como operando en la siguiente instrucción.

```
Entrada: x + y * 3
TAC generado:
  t0 = y * 3      — term: y * 3
  t1 = x + t0     — exp: x + t0
```

Generación para *while*

```
while (i < 10) { i++; }
TAC:
  L0:
    t0 = i < 10
    if not t0 goto L1
    t1 = i + 1
    i = t1
    goto L0
  L1:
```

Generación para *for*

El for se descompone en: inicialización + etiqueta + condición + cuerpo + incremento + salto.

```
for (int i = 0; i < 5; i++) { x = x + i; }
TAC:
  i = 0
  L0:
    t0 = i < 5
    if not t0 goto L1
    t1 = x + i
    x = t1
    t2 = i + 1
    i = t2
    goto L0
  L1:
```

Generación para if-else

```
if (x > 0) { y = 1; } else { y = 0; }
TAC:
  t0 = x > 0
  if not t0 goto L0
    y = 1
    goto L1
  L0:
    y = 0
  L1:
```

Generación para funciones

Las funciones usan el modelo *push/pop.* Al llamar, los argumentos se pasan con 'param' (equivale a un *push*). Al definir la función, los parámetros se reciben con *pop* – como antes habíamos "apilado" los argumentos, ahora la función necesita tomarlos. De esta manera los valores de los argumentos llegan a las variables locales de la función.

¿Por qué se hace así?

En lenguajes de alto nivel como C, simplemente escribimos *suma(3,5),* pero el procesador de nuestra computadora no entiende esto. Los procesadores reales (utilizando *Assembler)*manejan los argumentos exactamente de esta manera: empujando datos a la memoria *(push)* y sacándolos *(pop)*.

El código *TAC* está simulando este comportamiento real del hardware de una manera simplificada, para que el último paso del compilador (traducir esto a *Assembler*) sea fácil de programar.

```
int suma(int a, int b) { return a + b; }
int r = suma(3, 5);
TAC:
  FUNC suma:
    pop a
    pop b
    t0 = a + b
    return t0
  END FUNC suma
  param 3
  param 5
  t1 = call suma
  r = t1
```

Optimización de código *TAC*

TACOptimizer.py implementa cuatro pasadas de optimización que se ejecuta iterativamente hasta alcanzar un p ex unto fijo (sin más cambios) o un máximo de 10 iteraciones. Cada pasada opera sobre la lista de instrucciones *TAC* como *strings*.

Plegado de constantes

Evalúa operaciones donde ambos operandos son constantes numéricas en tiempo de compilación, reemplazando la operación por su resultado. Esto elimina cálculos que se pueden resolver estáticamente.

```
Antes:                    Después:
  t0 = 3 + 5      →       t0 = 8
  t1 = 10 * 2      →       t1 = 20
  t2 = 7 % 3       →       t2 = 1
```

Operadores soportados: +, -, *, /, %. La división por cero no se evalúa (se mantiene la instrucción original). Los resultados se convierten a entero si el resultado es un número entero exacto.

Implementación: Se usa una expresión regular para detectar el patrón 'var = num op num'. Se evalúa con la función *eval()* de *Python* y se reemplaza la instrucción por 'var = resultado'.

Propagación de Copias

Cuando un temporal se define como una copia simple (tN = x), se reemplaza cada uso posterior de tN por x directamente. Solo se propagan temporales que se asignan exactamente una vez para garantizar la corrección.

```
Antes:                    Después:
  t0 = x                  t0 = x
  t1 = t0 + 1     →       t1 = x + 1
  y = t1                  y = t1
```

Algoritmo en 3 pasadas

- **Pasada 1:** Identificar copias simples (tN = x) con *regex* (secuencia de caracteres que forma un patrón de búsqueda).
- **Pasada 2:** Verificar que los temporales no se reasignan (conteo de asignaciones = 1)

- **Pasada 3:** Reemplazar usos de temporales con sus valores usando *word boundaries* (\b) – El símbolo \b indica que el patrón debe ser una palabra completa, no parte de otra. Ejemplo:
    - *Regex:* \b**t1**\b
    - En **t1** + 5 → Coincide (reemplaza t1).
    - En t11 → No coincide (protege a t11).

Eliminación de Código muerto

Elimina instrucciones que definen temporales (tN) que nunca son usados como operando en ninguna otra instrucción. Un temporal se considera "muerto" si se asigna pero nunca aparece en el lado derecho de otra asignación ni en instrucciones de control.

```
Antes:                    Después:

t0 = a + b                (eliminada - t0 no se usa)
t1 = c * d        →       t1 = c * d
x = t1                    x = t1
```

Protecciones:

- No se eliminan instrucciones de control: *if, goto, return, param, call, FUNC, END, pop, DECLARE.*
- No se eliminan etiquetas (líneas que terminan en ':').
- Solo se eliminan asignaciones a temporales (tN), no a variables del usuario.

Simplificación Algebraica

Detecta y simplifica patrones algebraicos redundantes y auto asignaciones:

Patrón	Simplificación	Regla
x = x	(eliminada)	Autoasignación
t = x + 0 / t = x - 0	t = x	Identidad aditiva
t = x * 1 / t = 1 * x	t = x	Identidad multiplicativa
t = x * 0 / t = 0 * x	t = 0	Elemento absorbente

Iteración hasta punto fijo

Las cuatro fases se ejecutan en orden secuencial dentro de un bucle. Si ninguna pasada realiza cambios, el optimizador ha alcanzado el punto fijo y termina. Esto es necesario porque una optimización puede habilitar oportunidades para otra. Por ejemplo, el plegado de constantes puede crear copias simples que luego copy propagation puede aprovechar.

Reporte y comparación

El optimizador genera dos reportes:

- **Reporte:** Ofrece una lista detallada de cada optimización aplicada con formato 'Tipo: antes → después'
- **Comparación:** Muestra métricas de líneas originales, optimizadas, eliminadas y porcentaje de reducción.

Resumen de Archivos del Proyecto

Archivo	Líneas	Descripción
compilador.g4	~190	Gramática ANTLR4 del lenguaje
App.py	~127	Punto de entrada, orquestador del pipeline
Escucha.py	~394	Listener: análisis semántico, gestión de scopes
SemanticValidator.py	~466	Validación de tipos, inicialización, uso
IntermediateCodeGenerator.py	~663	Visitor: generación de TAC
TACOptimizer.py	~440	Optimizador de TAC (4 pasadas)
ErrorManager.py	~155	Singleton: errores y advertencias
SymbolTableManager.py	~180	Facade sobre la tabla de símbolos
tablaDeSimbolos.py	~100	TS, Contexto, Variable, Funcion
StatsCollector.py	~110	Recolector de estadísticas
Caminante.py	~38	Visitor básico para recorrido del árbol
compiladorLexer.py	auto	Generado por ANTLR4
compiladorParser.py	auto	Generado por ANTLR4
compiladorListener.py	auto	Generado por ANTLR4
compiladorVisitor.py	auto	Generado por ANTLR4

Conclusión

El desarrollo de este compilador permitió comprender la complejidad detrás de la traducción de lenguajes de programación. El uso de *ANTLR4* facilitó significativamente la etapa de *parsing*, permitiendo enfocar los esfuerzos en la lógica semántica y la generación de código. Asimismo, la utilización de *Git* fue una pieza clave en el flujo de trabajo, funcionando como una red de seguridad que permitió retomar estados estables del sistema tras implementar modificaciones complejas que afectaban la estabilidad general.

El proyecto fue un reto que demandó un pensamiento lógico riguroso. Uno de los mayores desafíos técnicos fue la implementación del Generador de Código Intermedio y sus fases de optimización, particularmente la gestión de etiquetas para saltos en estructuras anidadas. No obstante, el uso del patrón Visitor resultó ser una estrategia eficiente y elegante para modularizar esta lógica, facilitando el cumplimiento de todos los objetivos propuestos.

Referencias

- ***Link* al repositorio de *GitHub*:** https://github.com/joacoContreras/DHS-JoaquinContreras_2025
- ***Tag* del *commit* final:** ExamenFinal - 4eddb82